

Ayudantía 8 - POO Invierno

Profesor: Cristhian Rabi

Ayudante: Paulo Araya

10 de Agosto de 2026

Singleton:

El patron Singleton se trata de solo poder instanciar un objeto de una clase, es lo más parecido que se tiene a una variable global dentro de tu programa.

Sirve principalmente para mantener un único estado dentro de tu dominio, ej: para acceder a las mismas ArrayList que guardan tus objetos.

```
1  public class Fiesta {
2      private static Fiesta instancia;
3      private ArrayList<String> invitados;
4
5      private Fiesta() {
6          invitados = new ArrayList<>();
7      }
8
9      public static Fiesta getInstancia() {
10         if (instancia == null) {
11             instancia = new Fiesta();
12         }
13         return instancia;
14     }
15
16     public void agregarInvitado(String nombre) {
17         invitados.add(nombre);
18     }
19
20     public ArrayList<String> getInvitados() {
21         return invitados;
22     }
23 }
24
25 // main
26 Fiesta a = Fiesta.getInstancia(); // ambos apuntan al mismo objeto
27 Fiesta b = Fiesta.getInstancia();
28 a.agregarInvitado("Ana");
29 System.out.println(b.getInvitados()); // [Ana]
```

Factory:

El patrón Factory centraliza la creación de objetos en un único lugar, delegando la decisión de qué clase específica instanciar según algún parámetro o condición.

Sirve para abstraer la lógica compleja de elección y evitar que el resto del programa dependa de clases concretas, ej: dado un `String` tipo de figura, se retorna un `Circulo`, `Cuadrado` o `Triangulo` sin que quien llama conozca los detalles.

```
1  public interface Figura {
2      double getArea();
3  }
4
5  public class Circulo implements Figura {
6      private double radio;
7      public Circulo(double radio) { this.radio = radio; }
8      public double getArea() { return Math.PI * radio * radio; }
9  }
10
11 public class Cuadrado implements Figura {
12     private double lado;
13     public Cuadrado(double lado) { this.lado = lado; }
14     public double getArea() { return lado * lado; }
15 }
16
17 public class FiguraFactory {
18     public static Figura crearFigura(String tipo, double param) {
19         switch (tipo) {
20             case "circulo": return new Circulo(param);
21             case "cuadrado": return new Cuadrado(param);
22             default: throw new IllegalArgumentException("Tipo desconocido: " +
23 tipo);
24         }
25     }
26
27     // main
28     Figura f = FiguraFactory.crearFigura("circulo", 2.0);
29     System.out.println(f.getArea());

```

Strategy:

El patrón Strategy permite definir una familia de algoritmos intercambiables en tiempo de ejecución, encapsulando cada uno en una clase distinta que implementa una misma interfaz.

Sirve para evitar cadenas de `if/else` cuando un comportamiento puede variar, ej: un `Personaje` puede cambiar su forma de atacar (`Espada`, `Arco`, `Magia`) simplemente asignando otra estrategia.

```
1  public class Personaje {
2      private EstrategiaAtaque ataque;
3
4      public void setAtaque(EstrategiaAtaque ataque) { this.ataque = ataque; }
5
6      public void ejecutarAtaque() { ataque.atacar(); }
7  }

```

```

8 // strategy =====
9
10 public interface EstrategiaAtaque {
11     void atacar();
12 }
13
14 public class AtaqueEspada implements EstrategiaAtaque {
15     public void atacar() { System.out.println("espada"); }
16 }
17
18 public class AtaqueArco implements EstrategiaAtaque {
19     public void atacar() { System.out.println("arco"); }
20 }
21
22 public class AtaqueMagia implements EstrategiaAtaque {
23     public void atacar() { System.out.println("magia"); }
24 }
25
26 // main =====
27 Personaje p = new Personaje();
28 p.setAtaque(new AtaqueEspada());
29 p.ejecutarAtaque(); // espada
30 p.setAtaque(new AtaqueMagia());
31 p.ejecutarAtaque(); // magia

```

Visitor:

El patrón Visitor permite agregar nuevas operaciones sobre un grupo de objetos sin modificar sus clases, separando el algoritmo de la estructura sobre la que actúa.

Sirve para realizar distintas acciones sobre cada elemento de una colección, ej: un Visitador que exporta a JSON, calcula el área o dibuja, aplicado a las distintas figuras de un listado sin tocar sus clases.

```

1 public interface Figura {
2     void aceptar(Visitador v);
3 }
4
5 public class Circulo implements Figura {
6     double radio;
7     public Circulo(double r) { radio = r; }
8     public void aceptar(Visitador v) { v.visitar(this); }
9 }
10
11 public class Cuadrado implements Figura {
12     double lado;
13     public Cuadrado(double l) { lado = l; }
14     public void aceptar(Visitador v) { v.visitar(this); }
15 }
16
17 public interface Visitor {

```

```

18     void visitar(Circulo c);
19     void visitar(Cuadrado c);
20 }
21
22 public class VisitorArea implements Visitor {
23     private static ArrayList<Double> areas = new ArrayList<>();
24
25     public void visitar(Circulo c) {
26         double area = Math.PI * c.radio * c.radio;
27         System.out.println("Area circulo: " + area);
28         areas.add(area);
29     }
30     public void visitar(Cuadrado c) {
31         double area = c.lado * c.lado;
32         System.out.println("Area cuadrado: " + area);
33         areas.add(area);
34     }
35
36     // agregar clases aqui...
37
38     private void mostrarPromedioAreas() {
39         double suma = 0;
40         for (double d : areas) {
41             suma += d;
42         }
43         System.out.println("Promedio total: " + suma / areas.size());
44         areas.clear();
45     }
46 }
47
48 // Uso: se agregan operaciones sin modificar las clases de las figuras
49 ArrayList<Figura> figuras = new ArrayList<>();
50 figuras.add(new Circulo(2.0));
51 figuras.add(new Cuadrado(3.0));
52
53 Visitor v = new VisitorArea();
54 for (Figura f : figuras) {
55     f.aceptar(v); // le pasamos el visitor
56 }
57 v.mostrarPromedioAreas();

```

Fácilmente podríamos agregarle a cada clase un `getArea()`, pero cuando se tiene un código con más de 30 tipos de figuras, es mucho mejor tener un solo lugar para trabajar.